

CNN Foundations

- Convolution,
- Spatial Arithmetic,
- Receptive Field

Study Pack:

- Notes
- Quiz
- A worked Example (Jupyter Notebook)
- Simulator (Gradio platform)

Michael Mizzi

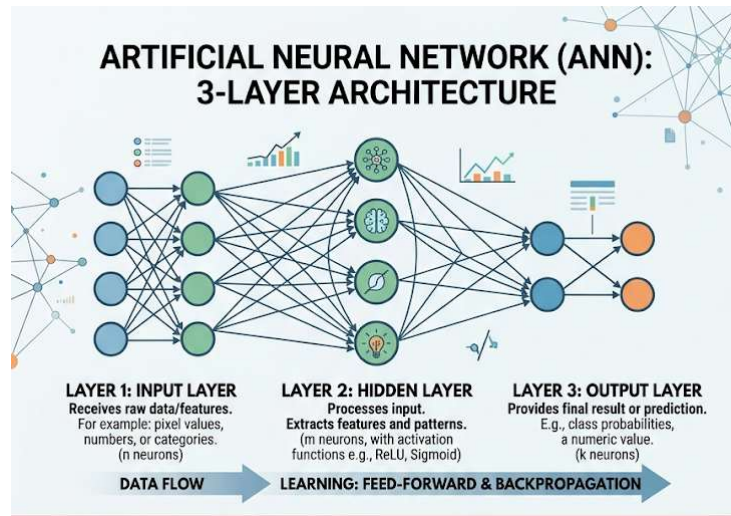
May 2026

Repository Link: https://github.com/micmizzi/ARI5118-Deep-Learning-For-Computer-Vision_MMIZZI.git



Why Traditional ANNs Fail?

Struggling with visual data



Example:

3-layer: $4i \rightarrow 4H \rightarrow 2o$ $\rightarrow$ 28x28 pixel: 3,150 parameters
128x128pixel: **65,550** parameters

4-layer: 784i/128H/64H/10o $\rightarrow$ 28x28 pixel: **~110k** parameters



01

Parameter Explosion

Small images need millions of weights, creating massive memory usage and overfitting risk.

02

Spatial Topology

Networks treat images as flat vectors, ignoring that nearby pixels are highly correlated.

03

Lack of Invariance

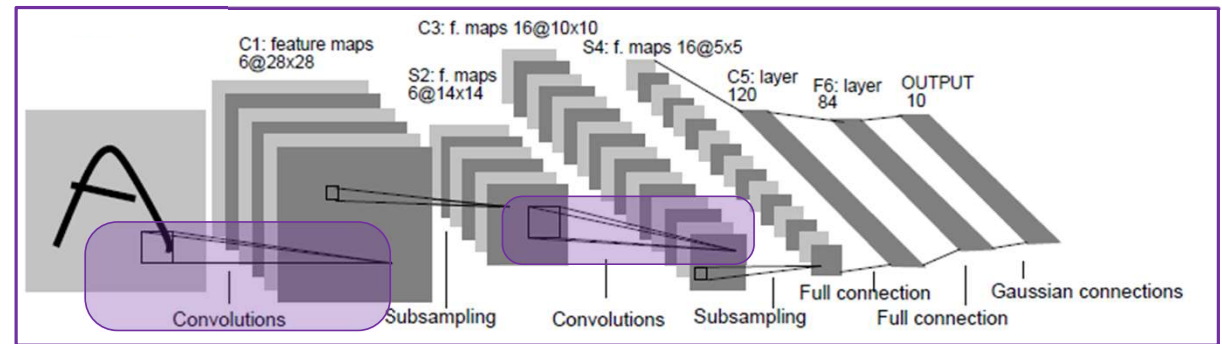
Networks lack translation invariance, failing to recognise objects shifted by a few pixels.

CNN: The core building blocks

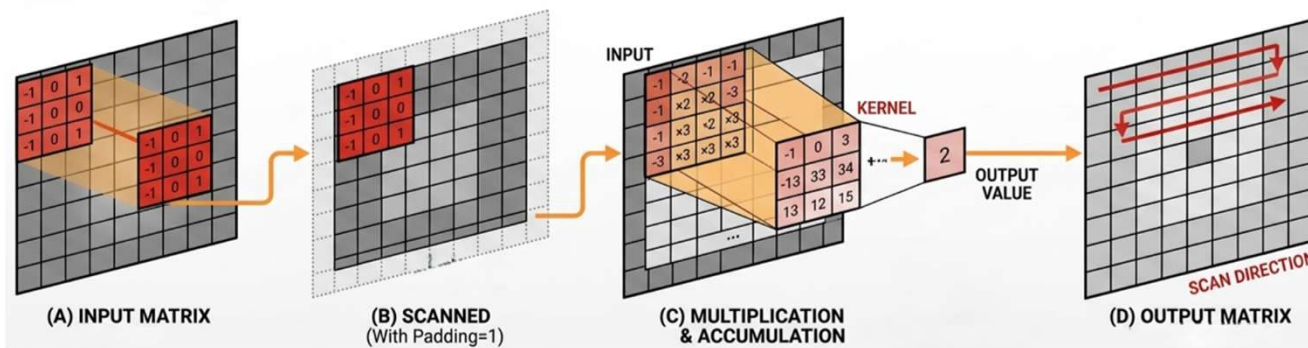
- From an 2D image to a 1D vector
- From input (i) to output (o)
- Convolutions
- Hyperparameters:
 - Kernel,
 - padding,
 - stride,
 - dilation
- What is the Effective Receptive Field?

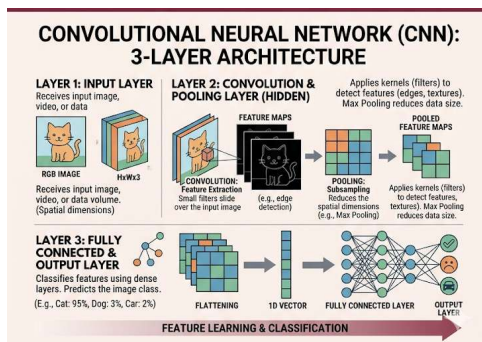
Architecture of LeNet-5, a CNN used for digit/character recognition

Gradient-Based Learning Applied to Document Recognition
Yann Lecun, L'eon Bottou, Yoshua Bengio, And Patrick Haffner
1998



~60,000 parameters





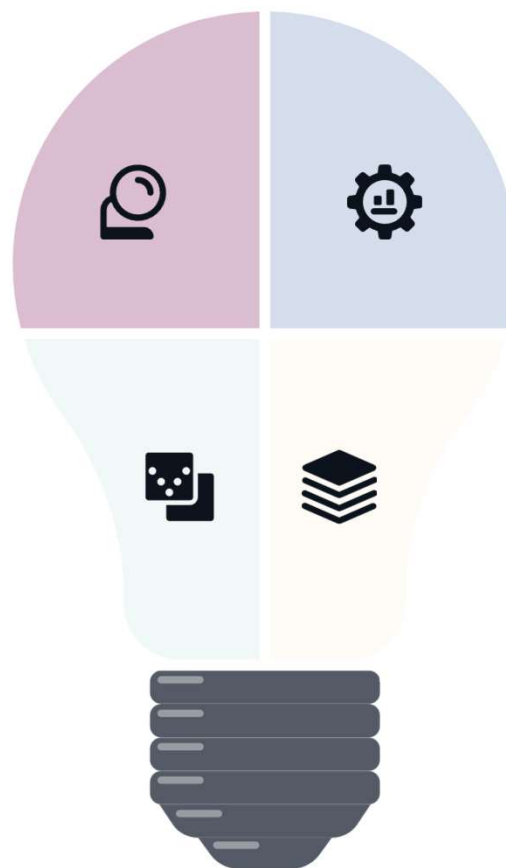
CNNs Mirror Biological Vision Through Local Connectivity

Local Vision

CNNs improve upon standard ANNs by focusing on small, local regions of input, effectively mimicking the human eye's ability to scan a scene for specific, immediate visual features.

Consistency

The same kernel weight is reused across the entire image, allowing the network to detect a specific feature consistently, regardless of its exact location in the input data.



Efficiency

By ensuring each neuron connects only to a small local input group rather than the whole image, CNNs achieve sparse connectivity, which drastically reduces total parameters.

Hierarchy

This structural design enables a sophisticated, efficient extraction pipeline, which progressively transforms raw pixel data into complex, high-level visual intelligence.

.....a side note

Input Matrix I:

```
[[6 3 7 4 6]
 [9 2 6 7 4]
 [3 7 7 2 5]
 [4 1 7 5 1]
 [4 0 9 5 8]]
```

Kernel K:

```
[[ -2  0]
 [  0 -1]]
```

Padded Input Matrix:

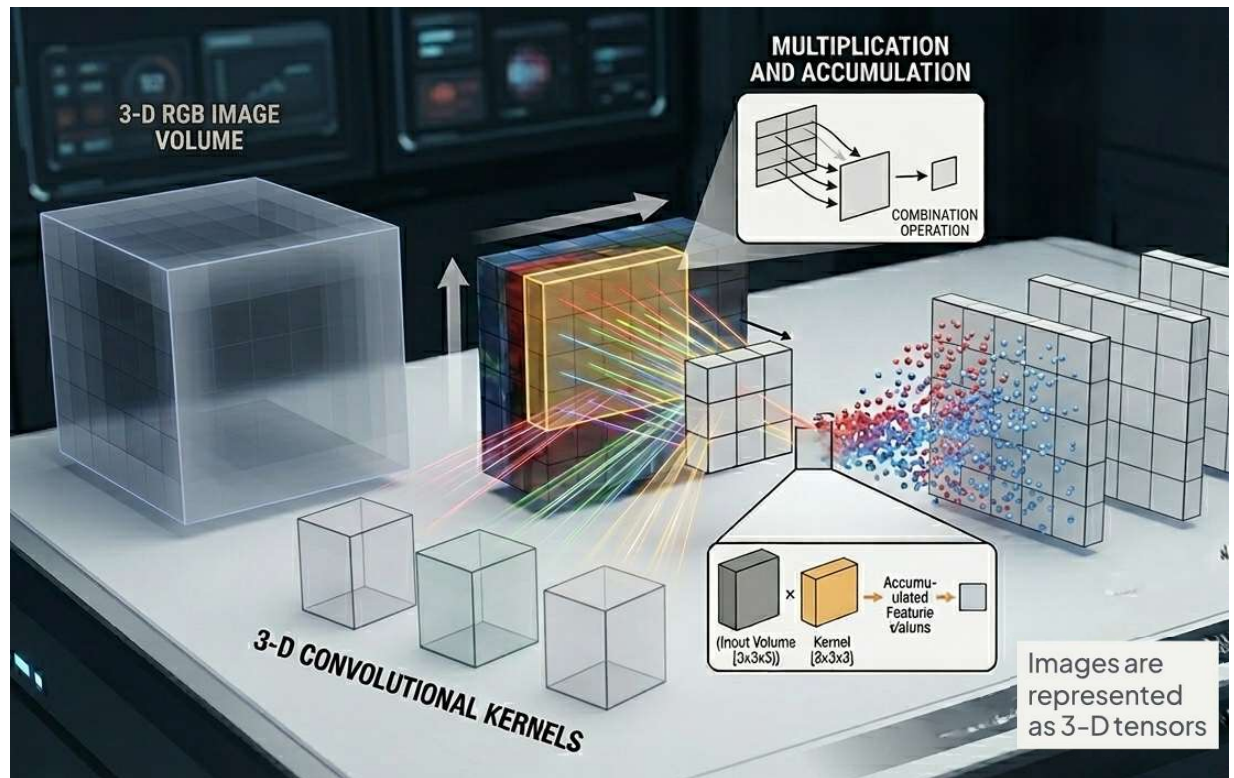
```
[[0 0 0 0 0 0 0]
 [0 6 3 7 4 6 0]
 [0 9 2 6 7 4 0]
 [0 3 7 7 2 5 0]
 [0 4 1 7 5 1 0]
 [0 4 0 9 5 8 0]
 [0 0 0 0 0 0 0]]
```

Effective Dilated Kernel:

```
[[ -2.  0.  0.]
 [  0.  0.  0.]
 [  0.  0. -1.]]
```

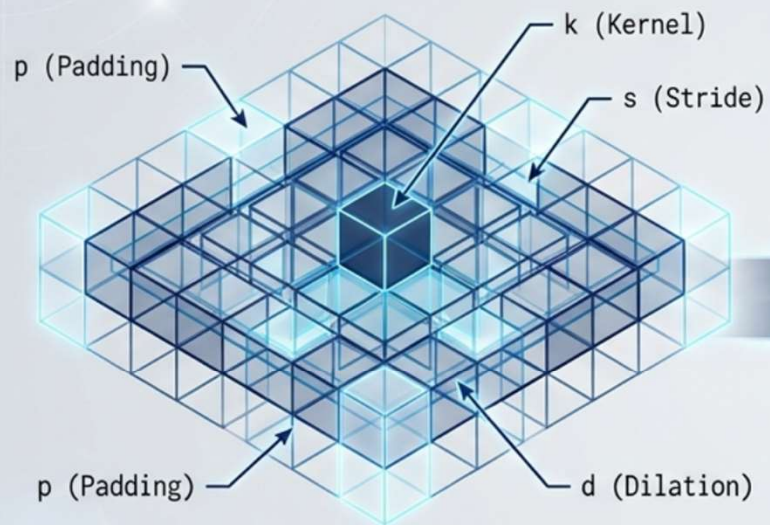
Final Output Matrix after Convolution:

```
[[ -2. -6. -7. -4.  0.]
 [ -7. -19. -8. -19. -8.]
 [ -1. -25. -9. -13. -14.]
 [  0. -15. -19. -22. -4.]
 [  0. -8. -2. -14. -10.]]
```



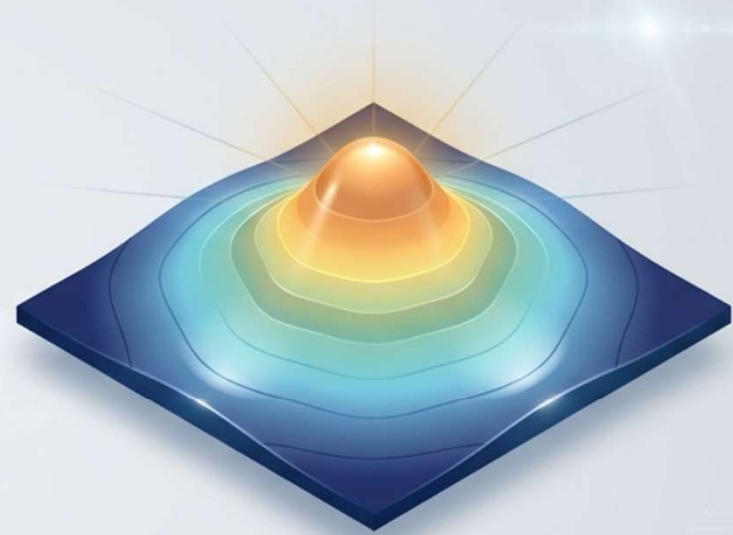
Reference to the Jupyter notebook "[walkthrough.ipynb](#)"

The Micro: Spatial Arithmetic



Master the pixel-level spatial operations that manipulate input matrices.

The Macro: Systemic Perception



Understand how these micro-operations compound to form the network's Effective Receptive Field.

Convolution Operation Mechanics

1

Sliding Window

A small array of weights, known as a kernel, moves across the input feature map at a set pace to scan the relevant regions systematically.

2

Multiplication

At each designated position, the kernel's individual weights are multiplied by the specific input units that they currently overlap directly.

3

Summation

These individual product results are summed together to produce a single definitive numerical value for that coordinate in the output map.

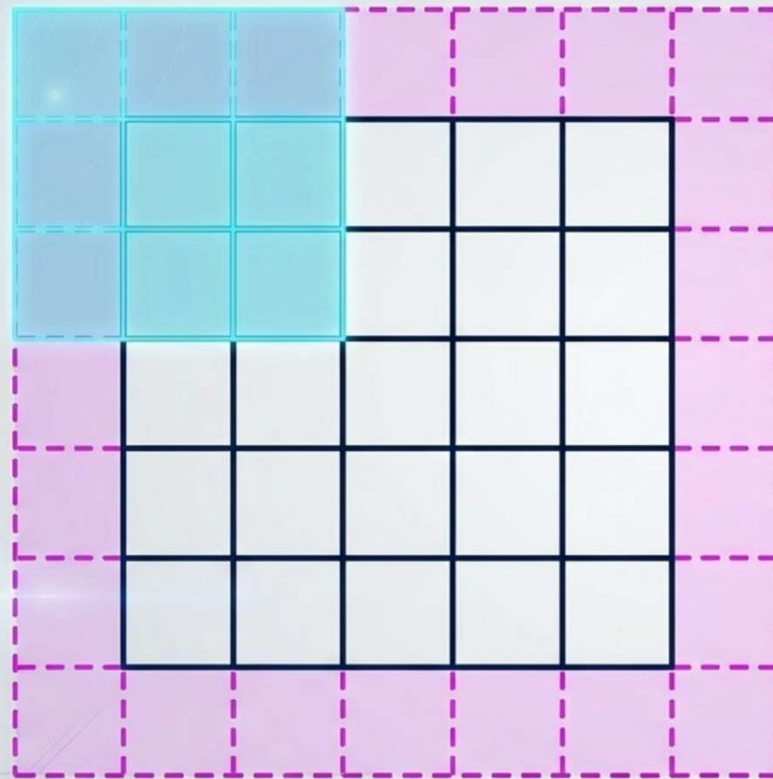
4

Integration

Distinct kernels are applied to each colour channel, and the resulting values are summed to create the final unified feature output map.

Padding

The Phantom Border: Preserving Resolution with Padding (p)



The Edge Penalty

In standard convolution, corner pixels are only sampled once, while central pixels are sampled multiple times. The image shrinks and edge data is lost.

The Solution

Padding the input with zeros artificially expands the effective input size to $i + 2p$.

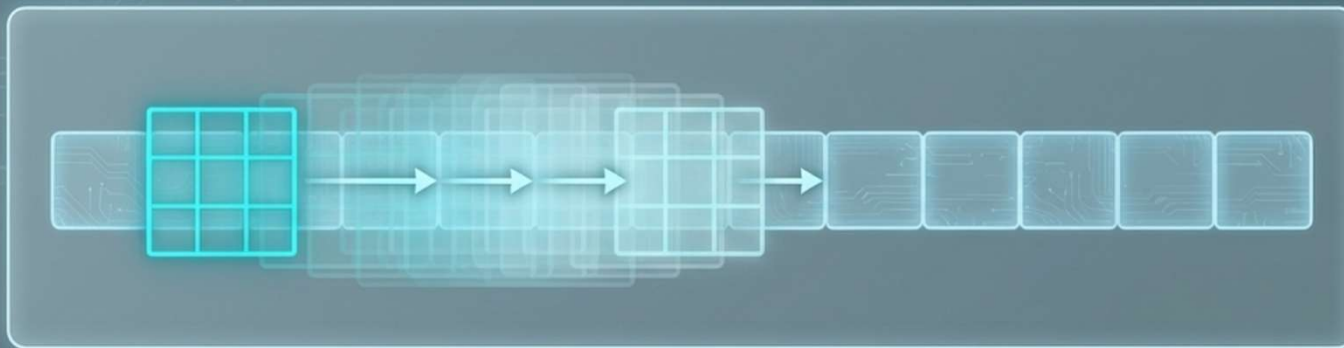
Same Padding

By setting $p = \text{floor}(k/2)$, the spatial output perfectly matches the input resolution.

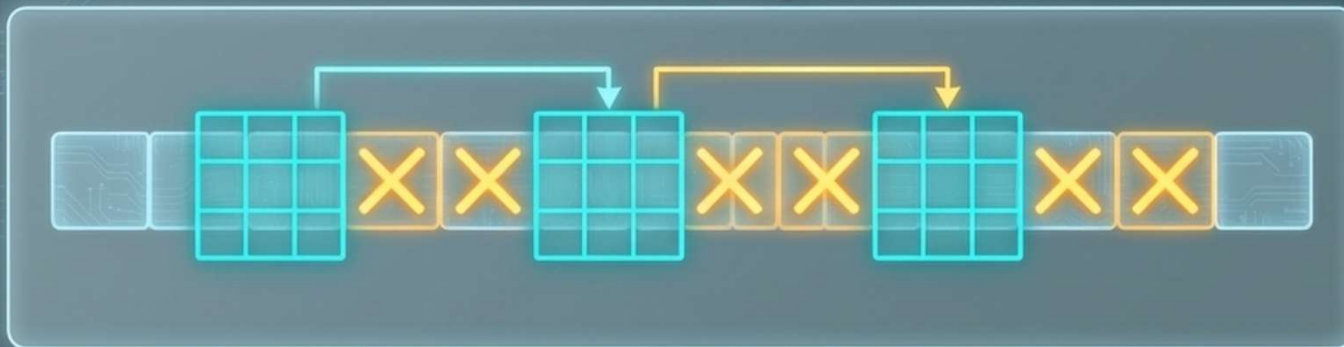
Stride

Adjusting the Focal Step: Subsampling with Stride (s)

Unit Strides ($s = 1$)



Strided Subsampling ($s = 2$)



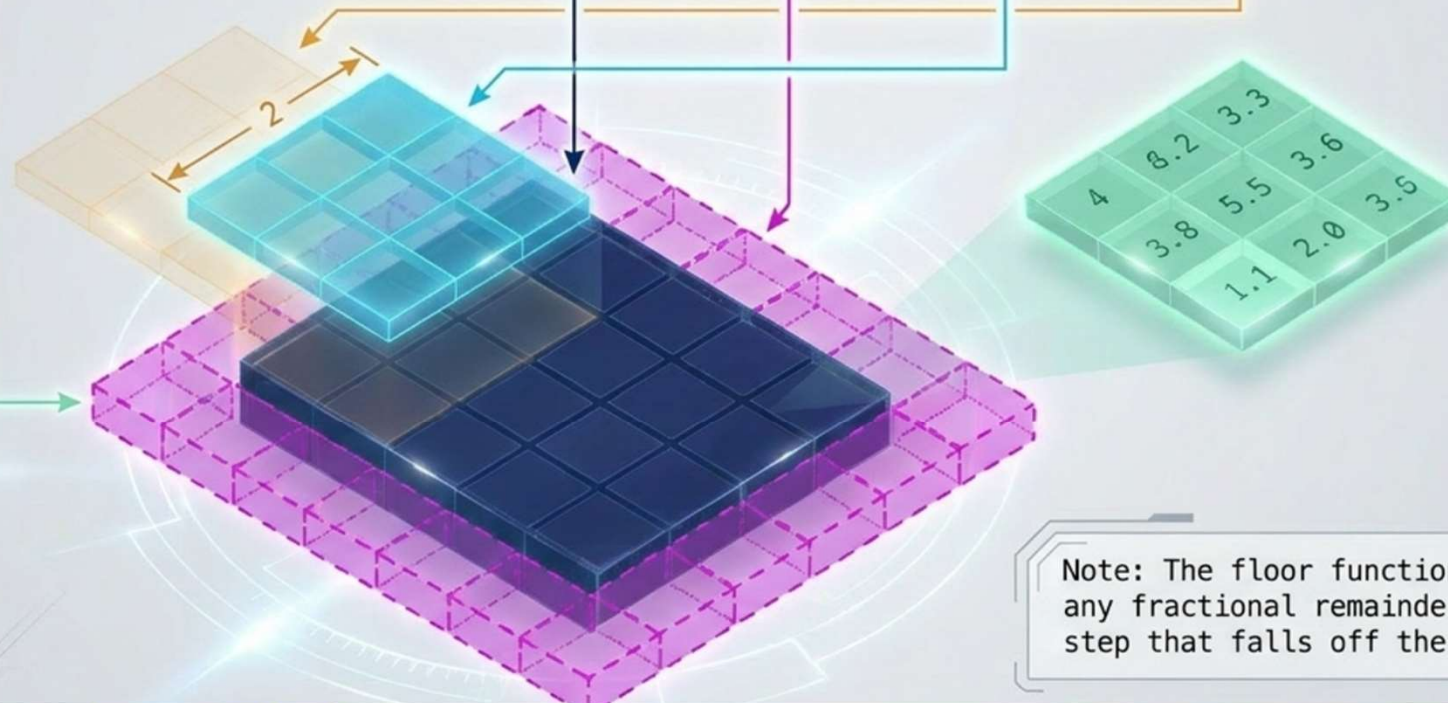
Efficiency

Strides ($s > 1$) constitute a form of spatial subsampling. Instead of translating by increments of 1, the kernel hops.

The Math

By dividing the spatial plane by s , the output dimensions shrink exponentially, reducing memory footprint and compute load deeper in the network.

$$o = \text{floor}((i + 2p - k) / s) + 1$$



Note: The floor function discards any fractional remainder of a stride step that falls off the grid.

Kernel Types

Kernels, or filters, serve as the trainable eyes of a neural network. These architectural components are essential for processing visual data through distinct mathematical operations that extract, expand, upscale, or compress features depending on the model's specific requirements.

Standard Kernels



These act as feature extraction workhorses, identifying oriented edges and simple corners.

Dilated Kernels



Also known as Atrous, these inflate by spacing weights to expand the receptive field.

Transposed Kernels



Used for upsampling in generation tasks, mapping small inputs into larger dimensions.

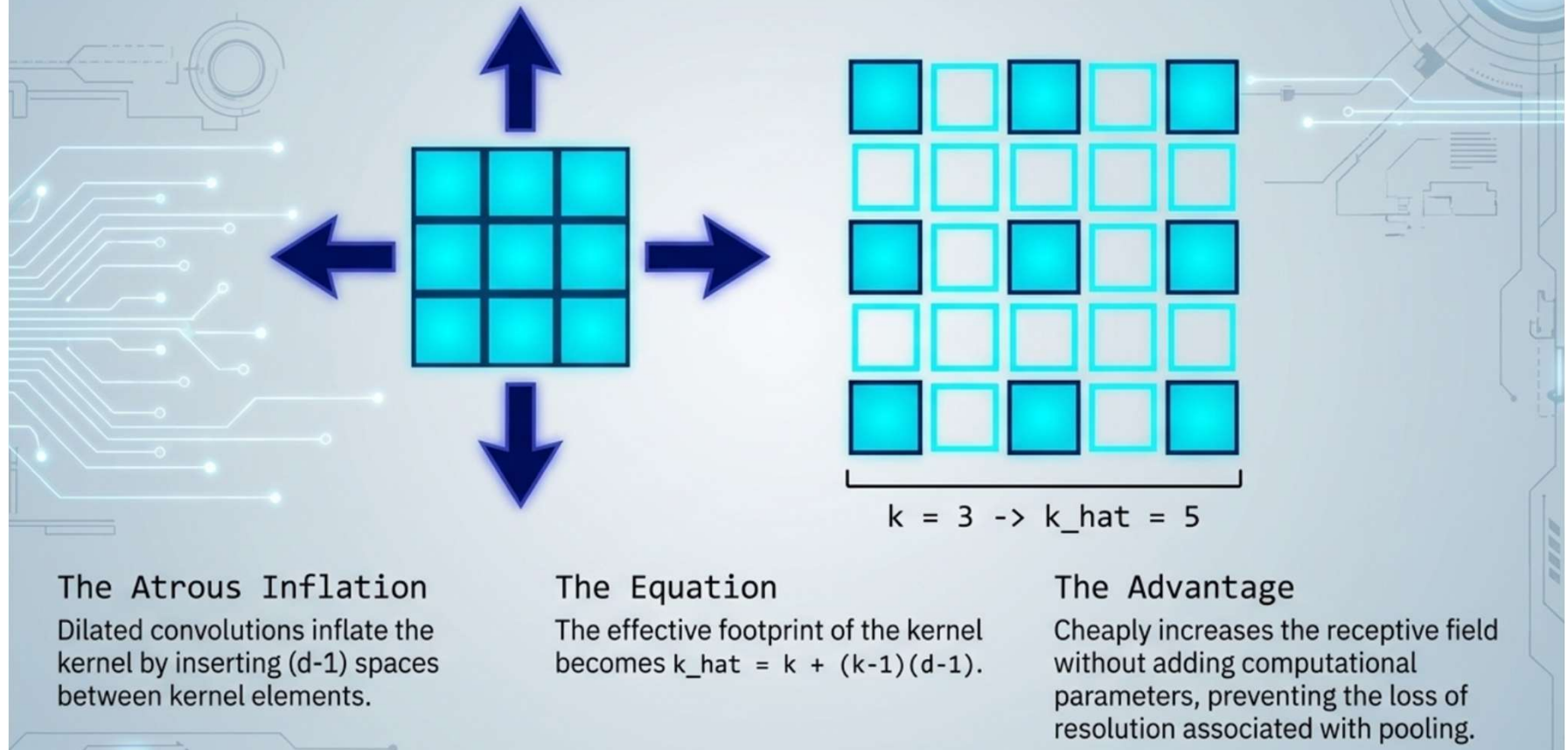
Separable Kernels



Decouple spatial and depth-wise operations, reducing footprint for compact applications.

Dilation

The Net Expansion: Multi-Scale Context via Dilation (d)



Mastering Spatial Dimension Arithmetic

CNN output dimensions rely on exact mathematical relationships between input size, kernel, padding, and stride. These variables govern information flow and computational efficiency within the network architecture.

1

Input Size (i)

3

Kernel Size (k)

1

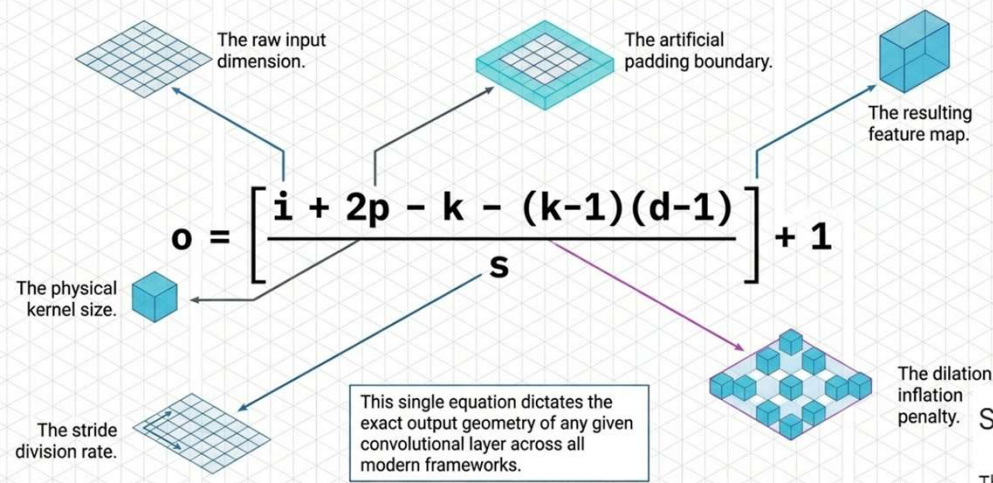
Padding (p)

2

Stride (s)

Compute Output Dimensions for Layers

The Universal Output Dimension Equation



- Convolution output: $(W - K + 2P) / S + 1$ per dimension; round down for noninteger
- Same padding: set P so output width equals input width when S is 1
- Valid padding: $P = 0$, output shrinks by kernel size minus 1
- Pooling output: use same formula as convolution with kernel K and stride S
- Stride > 1 reduces spatial size: larger S means fewer outputs
- Transpose convolution: inverse of conv—use output formula solving for output size
- Fully connected: output units equal specified neuron count, flatten spatial dims first
- Example check: Input 32×32 conv $K=5$ $S=1$ $P=0 \rightarrow 28 \times 28$ output
- Validation tip: always compute per-dimension and confirm channel counts after layers

Spatial Arithmetic

The Effective Kernel Size (K_{eff}) due to dilation ($D = 2$) is represented as follows:

$$K_{eff} = K + (K - 1)(D - 1) = 2 + (2 - 1)(2 - 1) = 3$$

The kernel behaves like a 3×3 filter where only the corners contain the weights:

$$\text{Effective Kernel} = \begin{bmatrix} -2 & \cdot & 0 \\ \cdot & \cdot & \cdot \\ 0 & \cdot & -1 \end{bmatrix}$$

Based on the above kernel and dilation adjustments the Output Dimensions (O) is equal to:

$$O = \frac{I + 2P - K_{eff}}{S} + 1 = \frac{5 + 2(1) - 3}{1} + 1 = 5$$

Thus the output is a 5×5 matrix.

Demo

Size: 28x28

Index: 29385

Source: MNIST Source: /kaggle/input/mnist-dataset/train-images-idx3-ubyte/train-images-idx3-ubyte

Blur (Box)

Kernel Size (N*N)

☒ 3 ☐ 5

```
[[0.111 0.111 0.111]
 [0.111 0.111 0.111]
 [0.111 0.111 0.111]]
```

PADDING

0 5

STRIDE

1 5

DILATION

1 5

Output 2

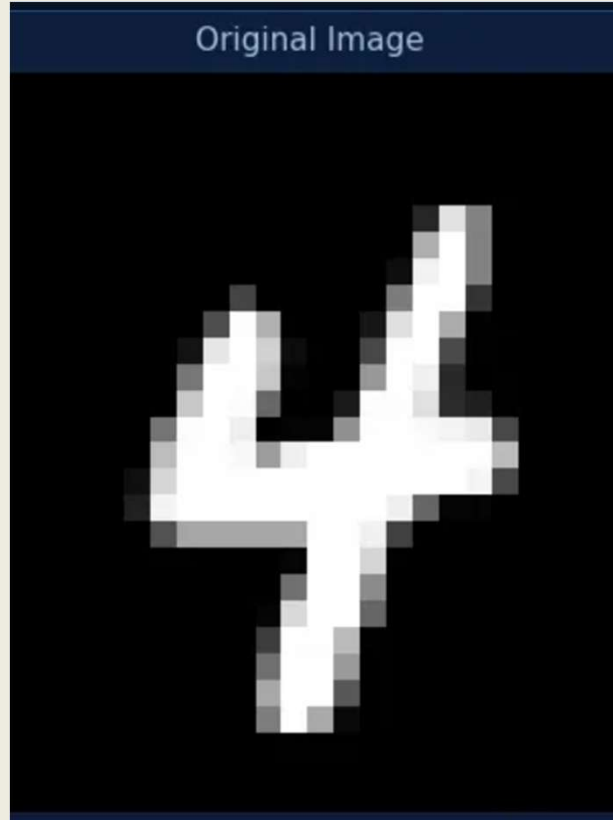
Kernel type: Blur (Box)



Setup 2 Details

CONVOLUTION SETUP 2 DETAILS

Output Size: (13, 13)
Receptive Field: 5x5



Sharpen

Kernel Size (N*N)

☒ 3 ☐ 5

```
[[ 0. -1.  0.]
 [-1.  5. -1.]
 [ 0. -1.  0.]]
```

PADDING

0 5

STRIDE

1 5

DILATION

1 5

Setup 1 Details

CONVOLUTION SETUP 1 DETAILS

Output Size: (13, 13)
Receptive Field: 5x5

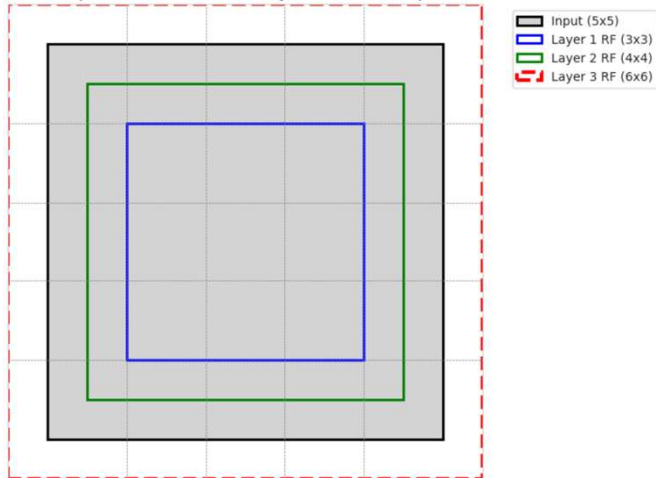
Output 1

Kernel type: Sharpen



Expand the Receptive Field with Dilation

Receptive Field Growth in a 3-Layer CNN (on 5x5 Input)



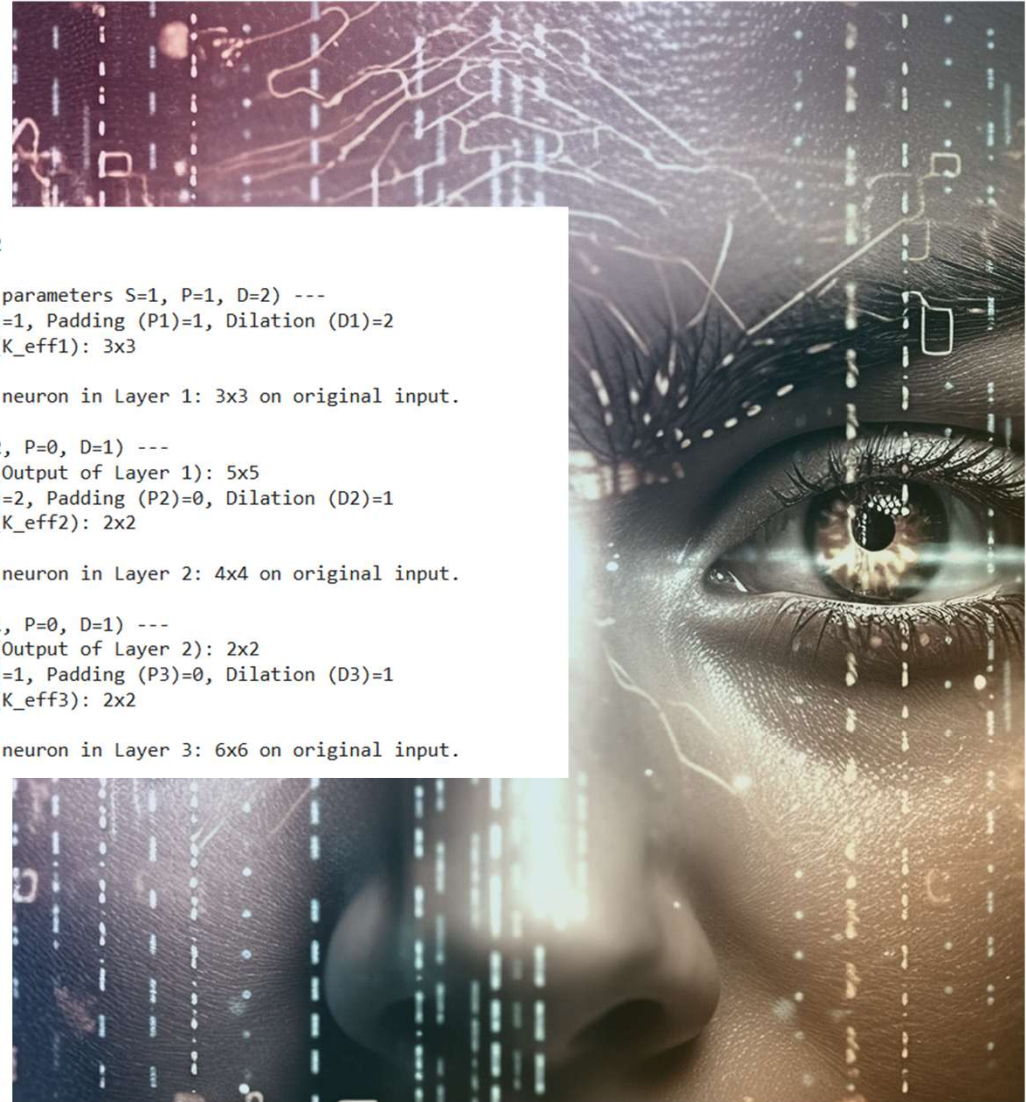
Original Input Size: 5x5
Original Kernel Size: 2x2

--- Layer 1 (using setup parameters $S=1$, $P=1$, $D=2$) ---
Parameters: Stride (S_1)=1, Padding (P_1)=1, Dilation (D_1)=2
Effective Kernel Size (K_{eff1}): 3x3
Output Size (O_1): 5x5
Receptive Field of one neuron in Layer 1: 3x3 on original input.

--- Layer 2 (assuming $S=2$, $P=0$, $D=1$) ---
Input Size to Layer 2 (Output of Layer 1): 5x5
Parameters: Stride (S_2)=2, Padding (P_2)=0, Dilation (D_2)=1
Effective Kernel Size (K_{eff2}): 2x2
Output Size (O_2): 2x2
Receptive Field of one neuron in Layer 2: 4x4 on original input.

--- Layer 3 (assuming $S=1$, $P=0$, $D=1$) ---
Input Size to Layer 3 (Output of Layer 2): 2x2
Parameters: Stride (S_3)=1, Padding (P_3)=0, Dilation (D_3)=1
Effective Kernel Size (K_{eff3}): 2x2
Output Size (O_3): 1x1
Receptive Field of one neuron in Layer 3: 6x6 on original input.

Increase spatial reach while keeping kernel parameters stable



RF vs. ERF

Distinguishing between the theoretical reach of a neural connection and its true effective impact on the final output.

Gaussian Influence

A concise subheading that supports the main message.

- ✘ The Effective Receptive Field follows a Gaussian distribution, ensuring that central input pixels exert significantly greater influence than those at the outer periphery.

$O(\sqrt{n})$ Scaling

A concise subheading that supports the main message.

- ✘ While theoretical receptive fields expand linearly at $O(n)$ with deeper layers, the effective field scales more conservatively at a rate of $O(\sqrt{n})$ as network depth increases.

Structural Effects

A concise subheading that supports the main message.

- ✘ Dilated convolutions and subsampling actively expand the effective field, whereas skip-connections, as seen in ResNets, can shrink it by narrowing the neuron's focus.

Common Pitfalls

Improper Initialization of Dilated Modules:

Standard random initialization schemes are often ineffective for specialized modules using dilation (d). For these architectures, "identity initialization"—where layers initially pass input directly through—is frequently required to ensure that back-propagation can reliably harvest the contextual information provided by the dilated kernels.

Premature Resolution Loss from High Strides:

Using a high stride (s) or subsampling too early acts as a form of spatial subsampling that significantly reduces feature map resolution. While this increases the receptive field multiplicatively, it is a common pitfall in dense prediction tasks (like semantic segmentation) where maintaining high-resolution output is critical.

Successive Resolution Shrinking without Padding:

Failing to use adequate padding (p) causes the spatial dimensions of feature maps to shrink after each layer because kernels cannot be centered on edge pixels. In deep networks, this can result in the loss of important border information or cause feature maps to "disappear" entirely due to successive size reductions.

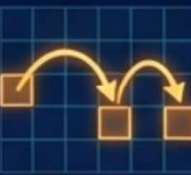
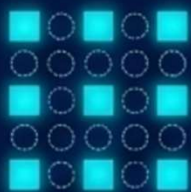
Inefficient Receptive Field Expansion:

Relying solely on increasing kernel size (k) to expand the "field of view" is computationally expensive, as parameters grow quadratically with k^2 while the receptive field only grows linearly. A more efficient approach often involves stacking deeper layers with smaller kernels or using dilation to grow the receptive field exponentially without increasing the number of weights.

Overestimating Effective Context from Theoretical Fields:

A major conceptual pitfall is assuming that the theoretical receptive field—calculated using k , s , and d —represents the actual area used by the network. In practice, the Effective Receptive Field (ERF) occupies only a small fraction of the theoretical area and decays in a Gaussian manner from the center, meaning the network "sees" much less of the global context than the architectural formulas might suggest.

Summary

Visual Icon	Operation & Purpose	Effect on Output Size	Example Use Case
	Standard Convolution: Local feature extraction.	Slight reduction ($i - k + 1$)	Early layer edge detection.
	Padded Convolution: Preserve spatial resolution.	Maintains exact size ($o = i$)	Deep ResNet intermediate blocks.
	Strided Convolution: Spatial downsampling & compression.	Reduces size by a factor of s	Replacing Max Pooling layers.
	Dilated Convolution: Wide context aggregation.	Expands receptive field without losing resolution.	Semantic Segmentation (WaveNet).

Next Steps.....

-Read through study notes
-follow the Walkthrough notebook
-visualize the impact of *kernel size/type, padding, stride and dilation* using the simulator
-take the quiz (cross check answer and refer to feedback)
-further reading available as required
(refer to study notes / folder in repository)

Further reading
Simulator
.gitattributes
LICENSE
README.md
ai_journal.txt
quiz_with_rationale.pdf
slides_draft.pdf
study_notes.pdf
walkthrough.ipynb

CNN Foundations ...the next building block ... **Pooling**